

كلية الهندسة المعلوماتية

جامعة دمشق

المقرر: برمجة تفرعية _ التقرير الأول للمشروع

اسم المشروع: محرك المعالجة عالي الأداء لنظام التجارة الإلكترونية
High-Performance E-Commerce Backend Engine

أسماء الغروب:

فرح مروان كبي

فاطمه فاضل الخطيب

تقى سمير البعلي

راما محمد عارف الدره

تيماء محمد بشار طرابلسي

فكرة المشروع وأهدافه (Project Concept & Objectives)

يأتي هذا المشروع كنموذج تطبيقي متقدم تحت عنوان "محرك المعالجة عالي الأداء لنظام التجارة الإلكترونية-High Performance E-Commerce Backend Engine)" كجزء من متطلبات مادة البرمجة المتوازية للعام الدراسي 2026.

يهدف المشروع بشكل رئيسي إلى إسقاط المفاهيم والنظريات العلمية التي تمت دراستها على مدار الجلسات الأكاديمية وتحويلها إلى حلول برمجية ملموسة. لا يهدف النظام إلى استعراض كثرة الواجهات أو تنوع الصفحات، بل يركز عمقاً على هندسة النظام الخلفي (Backend Architecture) لضمان استقراره وكفاءته من خلال:

- **ضمان سلامة البيانات: (Data Integrity)** منع تضارب العمليات المتزامنة وحل مشكلات Race Conditions عند تعديل المخزون.
- **إدارة وحوكمة الموارد: (Resource Management)** التحكم في سعة العمليات المتوازية لحماية النظام من الانهيار (Crash) دون التضحية بزمن الاستجابة.
- **المعالجة غير المتزامنة: (Asynchronous Processing)** فصل المهام الثانوية عن مسار الطلب الرئيسي لتقليل Latency.
- **معالجة البيانات الضخمة: (Batch Processing)** جدولة وهيكلة العمليات الكبيرة (مثل الجرد اليومي) على شكل دفعات (Chunks) لرفع الكفاءة.
- **توزيع الأحمال: (Load Distribution)** محاكاة بيئة خوادم متعددة لضمان التوافرية العالية (High Availability).

3. منهجية التقرير (Report Methodology)

لإثبات الجدوى الهندسية للتعديلات البرمجية التي تم إدخالها على النظام، يعتمد هذا التقرير على منهجية المقارنة التحليلية. (Before vs. After) سنقوم في كل قسم باستعراض سلوك النظام في حالته التقليدية (الخام)، ورصد نقاط الضعف والانهيار، ثم شرح الآلية البرمجية المتوازية المتبعة لحل المشكلة، وصولاً إلى قياس وتحليل النتائج بعد التعديل لإثبات الفارق في الأداء واستقرار النظام.

(الطلب الأول) حماية البيانات المشتركة من التضارب (Concurrent Access & Data Integrity):

1. قفل الأسطر الصارم (Row-Level Locking / Pessimistic Locking)

ما هو هذا المفهوم؟ هو آلية لمنع الخيوط المتوازية (Concurrent Threads) من التعديل على نفس السجل (Row) في قاعدة البيانات في نفس الوقت، حيث يتم إجبار أي عملية لاحقة على الانتظار حتى تنتهي العملية الحالية.

لماذا اخترناه؟ لأن القراءة العادية للمخزون كانت تسبب بيعاً زائداً (Over-selling) ورصيداً سالباً بالمخزن نتيجة قراءة عدة طلبات لنفس القيمة قبل تحديثها. هذا القفل يقضي تماماً على Race Condition ويضمن سلامة البيانات (Data Integrity).

أين تم تطبيقه في الكود؟ تم تطبيقه في ملف ConfirmOrderJob.php عند جلب بيانات الطلب والمنتج باستخدام الدالة: lockForUpdate();

```
$product = Product::where('id', $item->product_id)->lockForUpdate()->first();
```

2. المعاملات الذرية لقاعدة البيانات (Database Transactions)

ما هو هذا المفهوم؟ جميع مجموعة من عمليات قاعدة البيانات (قراءة، تعديل، حذف) تُنفَّذ ككتلة واحدة غير قابلة للتجزئة (Atomic)؛ إما أن تنجح كلها معاً أو تفشل كلها معاً. (All or Nothing)

لماذا اخترناه؟ لمنع حدوث "حالة غير متسقة" في النظام. فمثلاً لو احتوى الطلب على 3 منتجات، وتم خصم أول منتجين ثم انقطع الاتصال أو فشل خصم المنتج الثالث، تضمن ال Transaction التراجع التلقائي (Rollback) عن خصم المنتج الأول والثاني لكي لا تضيق حقوق المستخدم أو المتجر.

أين تم تطبيقه في الكود؟ تم تطبيقه في ملف ConfirmOrderJob.php بتغليف منطق المعالجة بالكامل داخل دالة القفل:

```
DB::transaction(function;{ ... } )
```

3. بوابات التداخل الذكية (Job Middleware - Without Overlapping)

ما هو هذا المفهوم؟ آلية تمنع نظام الطوابير من معالجة أكثر من مهمة (Job) تابعة لنفس المورد (نفس معرف الطلب Order ID) بشكل متزامن.

لماذا اخترناه؟ لحماية السيرفر وقاعدة البيانات من حدوث "الأقفال المتبادلة (Deadlocks)"، وتوفير الموارد من خلال تأجيل المهام المتطابقة بدلاً من جعلها تتصارع على نفس القفل في نفس اللحظة.

أين تم تطبيقه في الكود؟ تم تطبيقه في ملف ConfirmOrderJob.php عبر التابع ميديولوير:

```
(new WithoutOverlapping($this->order->id))->dontRelease()->expireAfter(10)
```

ثانياً: المفاهيم الخاصة بالطلب الثاني (إدارة الموارد والتحكم بالسعة):

1. معالجة المهام خلف الكواليس (Asynchronous Queuing)

ما هو هذا المفهوم؟ نقل العمليات الثقيلة والمنطق البرمجي المعقد خارج دورة حياة طلب ال HTTP الرئيسي (Request/Response Cycle)، وإسنادها لـ Background Workers.

لماذا اخترناه؟ لتحرير موارد السيرفر (مثل ممرات اتصال قاعدة البيانات وال Threads الخاصة بـ Apache/Nginx) فوراً. بدلاً من انتظار المستخدم 15 ثانية وتلقي خطأ Timeout سقوط السيرفر، يستقبل ردّاً فورياً بـ 45 ملي ثانية، بينما تدار الموارد في الخلفية بمرونة وثبات.

أين تم تطبيقه في الكود؟ تم تطبيقه في ملف OrderController.php عبر إرسال الطلب للطابور بدلاً من معالجته متزامناً:

```
ConfirmOrderJob::dispatch($order)->delay(now()->addSeconds(1;((
```

2. حوكمة محاولات الفشل (Tries Limit / Idempotency Control)

ما هو هذا المفهوم؟ وضع سقف محدد لعدد المرات التي يُسمح فيها للنظام بإعادة محاولة تنفيذ المهمة الفاشلة قبل إعلان فشلها نهائياً.

لماذا اخترناه؟ السلوك الافتراضي للطواير عند حدوث خطأ (مثل انقطاع الاتصال بقاعدة البيانات) هو إعادة المحاولة مراراً وتكراراً. بدون هذا القيد، سيدخل النظام في حلقة لانهائية (Infinite Loop) تستهلك كامل موارد المعالج والذاكرة وتؤدي لانهايار السيرفر. (Resource Exhaustion)

أين تم تطبيقه في الكود؟ تم تطبيقه في أعلى ملف ConfirmOrderJob.php عبر المتغير:

```
public $tries = 1;
```

3. . كسر الجمود وتحديد مهلة التنفيذ (Job Timeout Policy)

ما هو هذا المفهوم؟ وضع حد زمني أقصى لبقاء المهمة قيد التنفيذ داخل الـ Worker ، وإذا تجاوزته يتم قتله قسرياً.

لماذا اخترناه؟ لحماية النظام من "الخيوط العالقة" (Hung Threads) "إذا علق اتصال قاعدة البيانات لأي سبب، فإن الـ Timeout يضمن عدم بقاء الموارد محجوزة للأبد، مما يتيح للنظام استعادة أنفاسه وتميرير الطلبات الأخرى دون بطء وتراكم.

أين تم تطبيقه في الكود؟ تم تطبيقه في أعلى ملف ConfirmOrderJob.php عبر المتغير:

```
public $timeout = 5;
```

4. معالجة الاستثناءات الصارمة وحسم المصير (Defensive Exception Handling & Manual Failing)

ما هو هذا المفهوم؟ التقاط الأخطاء الحرجة (مثل سقوط قاعدة البيانات) بشكل استباقي عبر كتل try-catch وإجبار المهمة على الفشل المنظم فوراً.

لماذا اخترناه؟ لضمان الخروج الآمن (Graceful Degradation) عند حدوث كوارث برمجية؛ فبدلاً من ترك المهمة معلقة تستهلك الموارد بانتظار استجابة لن تأتي، نقوم بتوثيق الخطأ بالـ Logs ونقل الجواب فوراً لجدول الـ failed_jobs لإخلاء الذاكرة فوراً والحفاظ على استقرار السيرفر تحت ضغط JMeter.

أين تم تطبيقه في الكود؟

تم تطبيقه داخل تابع الـ `handle()` في ملف `ConfirmOrderJob.php`

ثالثاً: المفاهيم الخاصة بالطلب الثالث :

1. البرمجة غير المتزامنة وفصل مسار التنفيذ (Asynchronous Processing & Non-Blocking I/O)

ما هو هذا المفهوم؟ هو أسلوب في هندسة البرمجيات يسمح للنظام ببدء مهمة معينة ثم الانتقال فوراً لتنفيذ مهام أخرى دون انتظار انتهاء المهمة الأولى، مما يتيح تفرع مسارات التنفيذ. (Multi-path Execution Flows)

لماذا اخترناه؟ لأن المستخدم يحتاج فقط لمعرفة أن طلبه تم استقباله بسلام. أما حساب الضرائب المعقد، توليد الفواتير، وإرسال الإشعارات، فهي مهام "ثانوية" بالنسبة لزمن استجابة المستخدم، ونقلها إلى مسار تفرعي غير حابس (Non-Blocking) يرفع كفاءة النظام بشكل جذري.

أين تم استخدامه في الكود؟ تم استخدامه في `OrderController.php` عند استدعاء الجواب ونقله للطاير:

```
ConfirmOrderJob::dispatch($order)->delay(now()->addSeconds(1);((
```

وفي ملف ConfirmOrderJob.php حيث يتم تفريع المهمة الثالثة فوراً بعد نجاح المعاملة:

```
GenerateInvoiceJob::dispatch($order;)
```

2. نمط المنتج والمستهلك التفرعي (Producer-Consumer Pattern)

ما هو هذا المفهوم؟ هو نمط تصميمي مترامن (Concurrency Design Pattern) يقوم فيه خيط أو محرك (Producer) بإنشاء المهام ووضعها في مخزن مشترك (Queue)، بينما تقوم خيوط خلفية مستقلة تماماً (Background Workers / Consumers) بجلب هذه المهام ومعالجتها بالتوازي وبعيداً عن خيط المعالجة الرئيسي.

لماذا اخترناه؟ لتحقيق مبدأ الـ **Loose Coupling** بين واجهة النظام الخلفي (API) ومحركات المعالجة الثقيلة. يتيح هذا النمط موازنة الضغط على السيرفر؛ فحتى لو ضخ JMeter آلاف الطلبات في ثانية واحدة، يتم تخزينها بأمان في الطابور، ليقوم الـ Workers بمعالجتها بالتدرج حسب السعة المتاحة دون انهيار.

التقليدي إلى فئة Controller أين تم استخدامه في الكود؟ تم تطبيقه بتحويل كود بناء الفاتورة بالكامل من الـ مستقلة مدعومة بالطوابير في ملف

GenerateInvoiceJob.php:

```
class GenerateInvoiceJob implements ShouldQueue{ ... }
```

3. الحوكمة التفرعية وجدولة المهام (Throughput Governance & Rate Limiting)

ما هو هذا المفهوم؟ وضع قيود برمجية للتحكم في سلوك الخيوط التفرعية في الخلفية لضمان عدم استهلاكها لكامل موارد النظام بشكل مفرط. (Race for Resources)

لماذا اخترناه؟ لأن المهام التفرعية غير المتزامنة إذا تُركت تعمل بدون حوكمة عند حدوث ضغط عالٍ، قد تفتح مئات الخيوط في الخلفية مما يتسبب في اختناق المعالج. قمنا بتحديد متغيرات الحوكمة لمنع هذا السيناريو.

أين تم استخدامه في الكود؟ تم تطبيقه في أعلى ملف `GenerateInvoiceJob.php` من خلال تحديد ضوابط المهلة وال `Tries` لل `Worker` التفرعي:

`public $tries = 3` ; و `public $timeout = 10` ;

رابعاً: المفاهيم الخاصة بالطلب الرابع (معالجة البيانات الضخمة على دفعات Batch -

:(Processing):

1. تقطيع وتجزئة الذاكرة المتوازية (Memory Chunking & Data Streaming)

ما هو هذا المفهوم؟ هو آلية متقدمة في البرمجة المتوازية للتعامل مع البيانات الضخمة (Big Data) ، حيث يتم تقسيم طابور البيانات المستهدفة من قاعدة البيانات إلى كتل أو مجموعات صغيرة محددة الحجم (Chunks/Batches) وتمريها تتابعاً إلى خيط المعالجة، بدلاً من تحميل كامل البيانات دفعة واحدة.

لماذا اخترناه؟ لأن جلب آلاف السجلات من قاعدة البيانات دفعة واحدة عبر تجميع عادية (`get()`) يتسبب في اختناق واستهلاك كامل الذاكرة العشوائية (RAM Memory Exhaustion) ، مما يقود حتماً لانهايار السيرفر بخطأ (Fatal Error: Allowed Memory Size Exhausted). استخدام ال `Chunking` يضمن سقفاً ثابتاً ومستقراً لاستهلاك الذاكرة مهما تضاعف حجم البيانات وضغط ال `JMeter`.

أين تم استخدامه في الكود؟ تم استخدامه في ملف `DailySalesReportJob.php` حيث نقوم بجلب ومعالجة طلبات اليوم على دفعات تتسع كل منها ل 100 طلب فقط:

```
Order::where('status', 'confirmed')
```

```
->whereDate('created_at', $today)
```

```
->chunk(100, function ($ordersBatch) use (&$totalOrdersCount, &$totalRevenueSum) { ... });
```


2المعالجة الخلفية المجدولة(Scheduled Asynchronous Batch Execution)

ما هو هذا المفهوم؟ هو فصل مهمة المعالجة التحليلية الثقيلة (Analytical Batch Processing) زمنياً ومكانياً عن مسار عمليات ال HTTP الحية، وإسنادها إلى خيوط خلفية مستقرة تعمل بشكل دوري (Scheduled Background Tasks) دون التداخل مع تجربة المستخدم الفورية.

لماذا اخترناه؟ عمليات جرد المبيعات وحساب الأرباح اليومية هي عمليات إحصائية معقدة ومستهلكة للوقت. تفعيل هذا النمط يتيح لـ ReportController إرجاع رد فوري للمستخدم بأن "العملية بدأت بالخلفية" بإنتاجية عالية جداً، ويترك المعالجة الشاقة لـ Worker مستقل تماماً يُنهي عمله دون حجز قنوات الاتصال الحية.

أين تم استخدامه في الكود؟ تم استخدامه في ملف ReportController.php داخل دالة reportAfter() حيث يتم تحويل الطلب فوراً إلى الخلفية بشكل غير متزامن:

```
DailySalesReportJob::dispatch();
```

وفي ملف DailySalesReportJob.php عبر إعداد الفئة لتنفيذ واجهة الجدولة الخلفية المستقلة:
PHP

```
class DailySalesReportJob implements ShouldQueue { ... }
```

3. حوكمة الخيوط التحليلية وعزل الموارد(Resource Isolation & Throttle Control)

ما هو هذا المفهوم؟ هو وضع قيود صارمة على زمن تنفيذ وحياة الخيوط البرمجية الخلفية التي تتعامل مع العمليات الكبيرة (Long-Running Tasks) لمنعها من احتكار موارد المعالج (CPU) وقاعدة البيانات لفترات طويلة (Hung Background Processes).

لماذا اخترناه؟ نظرًا لأن عمليات الجرد وال Batch Processing تمر عبر مئات السجلات، فإن أي بطء في الشبكة أو قاعدة البيانات قد يجعل الخيط معلقاً للأبد، مما يستهلك طاقة السيرفر بالتوازي. لذا قمنا بتخصيص مهلة تنفيذ ممتدة ومحاولات محدودة تناسب طبيعة معالجة البيانات الضخمة دون التسبب في تجميد النظام.

أين تم استخدامه في الكود؟ تم تطبيقه في أعلى ملف `DailySalesReportJob.php` عبر ضبط محددات الحوكمة الزمنية المخصصة للمهام الثقيلة:

PHP

```
public $tries = 2;
```

```
public $timeout = 60;
```

خامساً: المفاهيم الخاصة بالطلب الخامس (توزيع الأحمال: Load Distribution -)

الخوارزمية المستخدمة هي Least Connections

تم استخدام خوارزمية Least Connections لأنها ترسل الطلب إلى الخادم الأقل انشغالاً، مما يحقق توازناً حقيقياً في أنظمة التجارة الإلكترونية التي تحتوي على طلبات متفاوتة في وقت المعالجة. كما ورد في المحاضرة الخامسة، فإن هذه الخوارزمية هي الأفضل عندما يكون الحمل غير ثابت، وهذا ينطبق تماماً على المشروع. تمت محاكاة مليون طلب، مع عرض أول 100 طلب فقط لتجنب استهلاك الذاكرة، بينما تم توزيع باقي الطلبات فعلياً على الخوادم.

الخرج النهائي يوضح عدد الطلبات التي استقبلها كل خادم بعد توزيع مليون طلب. نلاحظ أن الخوارزمية قامت بتوزيع الحمل بشكل متوازن جداً، حيث يكون الفرق بين الخوادم طلب واحد فقط، وهذا يؤكد فعالية Least Connections في موازنة الأحمال.

```
}  
  
,("architecture": "Distributed System Simulation (Load Balancing"  
  
,("algorithm_used": "Least Connections"  
  
,("total_simulated_requests": 1000"  
  
,("final_servers_load_status"  
  
,({name": "Server-1 (High Spec)", "active": 12, "speed_factor": 5})  
  
,({name": "Server-2 (Medium Spec)", "active": 13, "speed_factor": 8})  
  
,({name": "Server-3 (Low Spec)", "active": 12, "speed_factor": 12})  
  
[  
  
{
```